

Final Report

CYC386 Secure Software Design and Development

Midterm Lab Exam - 48-Hour DevSecOps Security Sprint

Project: Flask To-Do Application Security Hardening

Course: CYC386

Semester: Spring 2026

Instructor: Engr. Muhammad Ahmad Nawaz

Abstract

This report presents the security engineering work completed for a Flask To-Do web application under a 48-hour DevSecOps sprint. The project objective was to identify and mitigate critical web risks aligned with OWASP Top 10, specifically Insecure Direct Object Reference (IDOR), Cross-Site Request Forgery (CSRF), and Clickjacking. The team performed protection needs elicitation, threat modeling using STRIDE, and risk scoring using CVSS v3.1. Security controls were implemented directly in application code and configuration. A CI/CD pipeline was built in GitHub Actions to run automated tests, static analysis (CodeQL), dynamic analysis (OWASP ZAP baseline), and a release gate for critical findings. Results demonstrate practical hardening of object access, request integrity, and browser security headers, improving the application's resistance to common web attacks while maintaining development workflow quality.

1. Introduction

Modern web applications often fail due to access control and request integrity weaknesses rather than cryptographic flaws. Even simple CRUD systems can expose severe vulnerabilities when object-level authorization and browser security controls are weak. This sprint treated the Flask To-Do app as a real pre-production candidate requiring rapid but reliable hardening.

- The target system supports:
- user registration and login
 - task creation, editing, listing, and deletion
 - session-based authentication

- Because each task is user-owned data, secure object access is critical. The project therefore prioritized:
1. verifying ownership before object operations (IDOR mitigation),
 2. enforcing anti-CSRF protection on state-changing requests,
 3. preventing framing-based UI attacks via response headers.

In addition to code-level fixes, the work integrated DevSecOps practices through Git branching, pull request workflow, automated security scans, and vulnerability gating in CI.

Screenshot Placeholder - Project Overview:

[Insert screenshot of app home/tasks page here]

2. Protection Needs Elicitation (PNE)

2.1 Assets

| Asset | Description | Protection Need |

---|---|---

| User Accounts | `username`, `password\_hash`, authenticated identity | Confidentiality, integrity |

| Task Data | `title`, `completed`, `created\_at`, `user\_id` | Confidentiality, integrity, availability |

| Database | SQLite store (`todo.db`) for users/tasks | Integrity, availability |

| Session Cookies | Browser session for logged-in users | Confidentiality, integrity |

| CI/CD Workflow | Build, security scan, release gating | Integrity, traceability |

2.2 Threat Context

The app has multiple state-changing endpoints and user-owned objects. Main risk drivers:

- predictable object identifiers (`task\_id`) in URL paths
- form-based POST actions
- browser-executed UI in potentially hostile web contexts

2.3 Elicited Security Requirements

1. Enforce object-level authorization using authenticated owner identity on each task operation.
2. Protect all unsafe requests with CSRF tokens validated server-side.
3. Set anti-clickjacking headers for all HTTP responses.
4. Harden session cookie behavior (`SameSite`) to reduce cross-site abuse.
5. Run SAST + DAST automatically on every push/PR.
6. Fail pipeline on critical security findings.

****Screenshot Placeholder - PNE Artifact:****

`[Insert screenshot of PNE\_Report.md section here]`

3. STRIDE + DFD + Attack Tree

3.1 STRIDE Summary

| STRIDE | App-Specific Threat | Impact | Mitigation |

---|---|---|---

| Spoofing | Unauthorized login/session misuse | Account compromise | Password hashing, authenticated routes |

| Tampering | Foreign task edit/delete via IDOR | Data integrity loss | User-scoped task lookup (`id` + `user\_id`) |

| Repudiation | User denies sensitive action | Forensic gap | (Future) add audit logging |

| Information Disclosure | Cross-user task exposure | Privacy breach | Ownership checks + per-user filtering |

| Denial of Service | Endpoint flooding | Service degradation | (Future) rate limiting |

| Elevation of Privilege | User performs unauthorized actions | Access boundary bypass | Object authorization enforcement |

3.2 Data Flow Diagram (Textual)

```text

Browser

-> Auth routes (/register, /login, /logout)

-> Task routes (/ , /tasks/create, /tasks/<id>/edit, /tasks/<id>/delete)

Flask App (Blueprints + Flask-Login + CSRFProtect + response headers)

-> SQLAlchemy Models (User, Task)

-> SQLite DB (todo.db)

```

3.3 Attack Tree (Unauthorized Task Access)

```text

Goal: Access/modify another user's task

OR

- Manipulate task\_id in edit/delete endpoint (IDOR)
- Trigger victim POST via malicious site (CSRF)
- Frame UI and trick clicks (Clickjacking)

```

****Screenshot Placeholder - Threat Model Evidence:****

`[Insert screenshot of THREAT\_MODEL.pdf or app/THREATMODEL.md here]`

4. CVSS Risk Assessment (v3.1)

| Vulnerability | Base Score | Severity | Rationale |

|---|---:|---|---|

| IDOR | 6.5 | Medium | Authenticated attacker can target foreign objects by ID |

| CSRF | 8.8 | High | Victim-assisted unauthorized state changes across forms |

| Clickjacking | 5.4 | Medium | UI redress depends on user interaction |

These scores guided implementation priority: CSRF and IDOR were treated as immediate fixes; clickjacking was addressed via browser-layer hardening control.

****Screenshot Placeholder - CVSS Table:****

`[Insert screenshot of CVSS section from threat model/report here]`

5. Vulnerability Findings (IDOR / CSRF / Clickjacking)

5.1 IDOR

****Finding:**** Task object access risk existed if `task\_id` was handled without strict owner scoping.

****Exploitation scenario:**** Authenticated user attempts `/tasks/<foreign\_id>/edit` or delete.

****Security requirement:**** Query must include both target ID and authenticated user owner ID.

5.2 CSRF

****Finding:**** Form submissions are vulnerable if no anti-CSRF token validation is enforced.

****Exploitation scenario:**** Victim visits attacker page that auto-submits POST requests.

****Security requirement:**** Enable global CSRF validation and include hidden token in every unsafe form.

5.3 Clickjacking

****Finding:**** Browser can be tricked into clicking hidden controls if framing is allowed.

****Exploitation scenario:**** App embedded in attacker iframe with deceptive overlay.

****Security requirement:**** Add `X-Frame-Options: DENY` and CSP `frame-ancestors 'none'`.

****Screenshot Placeholder - Before/After Findings:****

``[Insert screenshot of findings table or issue tracker notes here]``

6. Security Implementation (Code-Level)

6.1 IDOR Fix

Implemented in ``app/routes/tasks.py``:

```
```python
def _get_user_task_or_404(task_id):
 return Task.query.filter_by(id=task_id, user_id=current_user.id).first_or_404()
```
```

Result: foreign task IDs no longer resolve for non-owners.

6.2 CSRF Fix

Implemented in ``app/__init__.py``:

```
```python
from flask_wtf.csrf import CSRFProtect
csrf = CSRFProtect()
csrf.init_app(app)
```
```

Implemented in templates (example):

```
```html
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```
```

Added in login/register/create/edit/delete/logout forms.

Session hardening in ``config.py``:

```
```python
SESSION_COOKIE_SAMESITE = "Lax"
```
```

6.3 Clickjacking Fix

Implemented in ``app/__init__.py``:

```
```python
@app.after_request
def add_security_headers(response):
 response.headers["X-Frame-Options"] = "DENY"
 response.headers["Content-Security-Policy"] = "frame-ancestors 'none'"
 return response
```
```

6.4 Additional Test Evidence

`tests/test\_security.py` includes:

- unauthenticated redirect check
- owner-only task visibility
- IDOR block on foreign task
- CSRF rejection on task create without token
- CSRF rejection on logout without token

****Screenshot Placeholder - Code Fixes:****

`[Insert screenshots of key code blocks from routes/templates/config here]`

7. CI/CD Pipeline Details

Pipeline file: `.github/workflows/ci-cd.yml`

7.1 Trigger Strategy

- On every `push`
- On every `pull\_request`

7.2 Jobs Implemented

1. ****Install Dependencies and Run Tests****
2. ****CodeQL SAST****
3. ****OWASP ZAP Baseline Scan****
4. ****Fail on Critical Vulnerabilities**** (gate)

7.3 Security Gate

The pipeline includes a gate that checks GitHub code scanning alerts and fails when open critical alerts exist for the repository.

7.4 Artifact Evidence

ZAP job uploads `zap-baseline-report` artifact containing report files and optional runtime log.

****Screenshot Placeholder - Workflow Runs:****

`[Insert GitHub Actions run screenshot with all jobs here]`

8. SAST + DAST Results (Before/After)

8.1 SAST (CodeQL)

****Before fixes:**** Access-control and request-integrity weaknesses were present in implementation design.

****After fixes:**** Code reflects user-scoped authorization, CSRF-protected forms, and anti-clickjacking headers. Critical SAST gate is active in CI.

8.2 DAST (OWASP ZAP Baseline)

****Before fixes:**** Expected higher browser/security header concerns and practical exposure around unsafe form actions.

****After fixes:**** Security headers and CSRF controls are in place; ZAP baseline reports are generated through CI artifact.

8.3 Test Run Evidence

Local test output confirms 5 security tests passing.

****Screenshot Placeholder - Security Testing Evidence:****

`[Insert screenshot of unittest output]`

`[Insert screenshot of CodeQL results page]`

`[Insert screenshot of ZAP artifact/report files]`

9. Conclusion and Future Hardening

This sprint achieved the required security objectives for the Flask To-Do app:

- IDOR mitigation via strict object ownership checks
- CSRF mitigation via global middleware and form tokens
- Clickjacking mitigation via response headers
- CI/CD security automation with SAST, DAST, and critical gating

The current system is significantly more attack-resistant than the baseline implementation and demonstrates practical alignment with CYC386 lab outcomes.

Future Hardening Recommendations

1. Add login rate limiting and abuse controls.
2. Add structured audit logging for auth and task events.
3. Disable `debug=True` in production execution path.
4. Add integration tests for full authenticated multi-user attack scenarios.
5. Expand DAST with authenticated scan profiles.

References

1. OWASP Top 10 - Web Application Security Risks.
2. CVSS v3.1 Specification Document.
3. Flask Security and Flask-WTF CSRF documentation.
4. GitHub Actions / CodeQL documentation.
5. OWASP ZAP Baseline Scan documentation.

Appendix A - Evidence Checklist (Fill During Submission)

- [] PR created and reviewer assigned (screenshot attached)
- [] Actions run with required jobs (screenshot attached)
- [] CodeQL result page captured
- [] ZAP artifact downloaded and attached
- [] `THREAT\_MODEL.pdf` present in repo root
- [] `Final\_Report.pdf` exported from this report
- [] Demo video recorded (5-7 minutes)